SABLE: Minimalist Instruction-Level Authenticated Encryption for Constrained Confidential Computing

Hamid Noori and Carlton Shepherd
Department of Computer Science
Durham University
Durham, United Kingdom
{hamid.noori, carlton.g.shepherd}@durham.ac.uk

arXiv:2607.16771v1 [cs.CR] 18 Jul 2026

Abstract

Conventional processor designs expose code and data as plaintext throughout execution, rendering them inherently vulnerable to attacks that recover intellectual property or modify security/safety checks. Instruction-level encryption (ILE) enables CPU-level decryption, execution, and optionally authentication of individual encrypted program instructions at runtime. However, existing proposals depend on specific micro-architectures, detect corrupted instructions after they have executed, rely on non-standard ciphers, or require complex analyses of program state. In this work, we introduce and present a design exploration of a RISC-V processor architecture (Sable) that enables minimally invasive instruction-level authenticated encryption of programs. Sable is agnostic to the underlying micro-architecture, remaining compatible with the standard RISC-V toolchain with minor changes to post-process compiled ELF binaries. We integrate a decrypt-and-verify stage at two points—the instruction-memory wrapper and the CPU frontend—and explore seven ILE micro-architectures from a single-cycle (combinational) design to six multi-cycle (sequential) variants. We implement and evaluate the designs using ASCON-128a on a Xilinx Artix-7 FPGA with the open-source NEORV32 system on chip. Relative to baseline performance, the configurations span LUT, performance, power and energy per instruction overheads of 1.6–9.3×, 4.1–10×, 1.5–8× and 10.4–80×, respectively, using the Dhrystone benchmarking suite. Finally, we discuss design trade-offs, highlighting area-, performance-, and energy-aware design points.

CCS Concepts

• **Security and privacy** → **Embedded systems security**; *Tamper-proof and tamper-resistant designs*; *Block and stream ciphers*; • **Hardware** → Reconfigurable logic and FPGAs.

Keywords

Instruction-level encryption, RISC-V, embedded systems security

1 Introduction

Embedded processors form the core of cyber-physical systems across manufacturing [19], vehicle control units [18], medical devices [37], and industrial control systems [13], where firmware performs safety-critical functions and embodies valuable intellectual property. Adversaries may target code memory to extract confidential firmware or to rewrite it so unauthorised code executes as if legitimate. This has been demonstrated through JTAG and debug-interface abuse, bus probing, and fault injection [26, 27, 34], and through bypasses of secure boot and read-out protection on Renesas [33], Volkswagen and BMW units [23], and the NVIDIA Tegra X2 used in the Tesla Autopilot and Mercedes-Benz infotainment systems [4]. Firmware extraction, in turn, has enabled product cloning and counterfeiting in practice: some real-world examples include the Hondata s300 ECU [29] and Allen-Bradley CompactLogix programmable logic controllers (PLCs) [32, 38].

In response, *instruction-level encryption* (ILE) conceals the instruction stream under a CPU-bound key [8, 12, 14, 22, 28]. Programs are provisioned in encrypted form and incrementally decrypted inside the processor, e.g., word by word, so attacks that successfully target program memory retrieve only encrypted instructions. Realisations differ in instruction set (ISA) and micro-architecture, cipher (e.g., PRINCE [17], XOR [16, 22], Trivium [12]), key management (e.g., PUF [14]), and where the decryption boundary is drawn, whether at the fetch [12] or decode [17] stage, cache level [8], or MMU page [22]. For sensitive devices, the boundary should reside as deep inside the processor as can be afforded.

Addressing attacks that modify or inject new words is more difficult, wherein adversarial words may still decrypt to pseudo-random opcodes with undefined side-effects or ones which are valid.[1] One line of work tackles this by deriving the cipher state that decrypts each instruction from those executed beforehand [10, 36]. These *implicit* approaches still enable the execution of unauthorised garbage opcodes, however. They also require pre-computing inter-instruction states, necessitating deep modifications to toolchains and whole-program control-flow graph analyses, with restrictions to specific pipelines and ISAs [10, 36]. Tackling this requires *explicit* authentication, i.e., verifying code-associated authentication tags that raise an immediate exception upon mismatch. Existing designs, however, amortise tags over blocks of instructions, restricting detection to block boundaries, and recover finer granularity only by using multiple layers of complexity [7, 25]. More fundamental points in this space—per-instruction, explicit, chaining-free authenticated decryption from a single primitive with loose coupling to toolchains and micro-architectures—remain underexplored.

In this paper, we present Sable (Single-layer Authenticated, Block-free Lightweight Execution), a RISC-V architecture that enables minimalist authenticated ILE of independent instructions. We realise Sable on the NEORV32 core at two integration points—the instruction-memory (IMEM) wrapper and the CPU frontend—in encryption-only and authenticated configurations, exploring the design space across seven implementations, from a single-cycle (combinational) design to six multi-cycle variants. To our knowledge, Sable is the first micro-architecture-agnostic, authenticated

[1] Under XOR-based ILE schemes [16, 22], ciphertexts are malleable; an attacker can modify targeted instruction bits without knowing the key.

ILE architecture that characterises the design space across area, performance and energy consumption considerations. We further extend the Spike RISC-V ISA simulator [35] to support authenticated encrypted execution, providing a functional reference model for verifying and prototyping designs at the ISA level.

1.1 Threat Model and Assumptions

We consider a conventional system-on-chip with a single processor, optionally including instruction and data caches. The processor core is considered to be trusted, encompassing the internal state, e.g., program counter, register file, and arithmetic logic units, where the execution is performed. We consider attackers capable of observing and modifying instructions stored in untrusted memory, e.g., IMEM, whether partially or in full, alongside bus signals and memory interfaces; and inject new unauthorised instructions into the instruction stream in untrusted elements (e.g., at the system bus). Attack scenarios include extracting program instructions to counterfeit firmware and compromise intellectual property, performing unauthorised code execution or modifying firmware in order to disable security and safety controls. We define the trust boundaries in Figure 1. Physical or micro-architectural attacks that target this region are thus considered out of scope. Sable focusses on instruction-level protection; control-flow integrity (CFI) of higher-level program semantics is not covered explicitly by Sable. Code reuse attacks are similarly not covered, e.g., return-to-libc [31] and ROP [24] and JOP [5] gadgets in the (encrypted) binary.

1.2 Related Work

Hardware ILE schemes divide into encryption-only and authenticated designs; Table 1 positions Sable against both groups by security and deployability considerations.

Encryption-only designs. Polyglot [28] combines AES and elliptic curve cryptography for page-level encryption, extending protection to OSs and hypervisors; however, its reliance on MMUs limits applicability to lightweight embedded systems. Morpheus [8] protects memory contents between cache levels using the QARMA cipher [2] but is evaluated only via gem5 simulation [3]. ASIST [22] integrates hardware support for ILE at instruction and block granularity using XOR and transposition, with a later extension introducing AES-based block decryption before the instruction cache [6]. A scheme based on the Trivium stream cipher [12] decrypts instructions during fetching, ensuring plaintext exists only within the core. Programs are divided into basic blocks with independent initialisation vectors (IVs) for greater granularity, though integrity verification is absent. PolEn [20] extends this with runtime polymorphism to mitigate side-channel analysis, while Morpheus II [11] tackles control-flow hijacking protection at instruction granularity on RISC-V. ECP [9] uses PRINCE-based encryption for program consistency but lacks hardware validation. Lee et al. chain an LFSR mask through previously stored instructions to bind firmware to a device against counterfeiting [16]; their later SEP uses PRINCE-encrypted basic blocks to the block address and pairs `CALL`/`RETURN` through explicit check instructions [17], gaining coarse-grained integrity at the cost of a custom ISA and assembler.

Authenticated designs. Explicit two-layer schemes pair encryption with a separate integrity mechanism at block granularity:

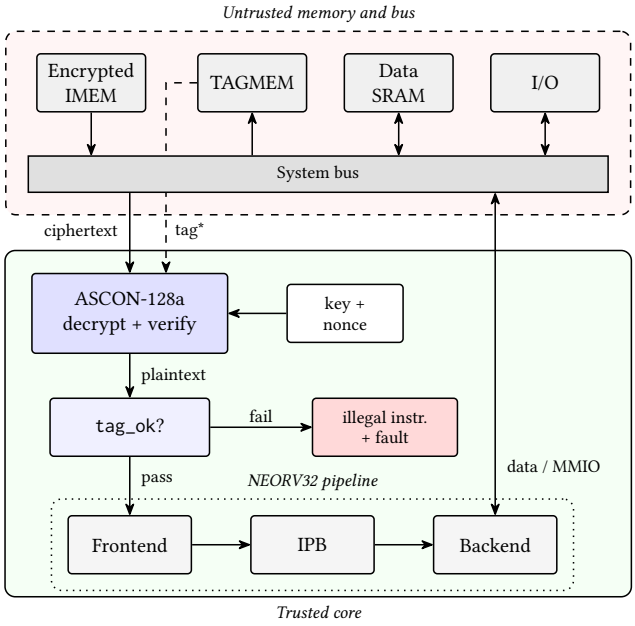


Figure 1: Sable block diagram (`FEAUDC` design; Section 2.1.2).
(*`TAGMEM` is directly connected to the ASCON module, not via the system bus.)

SOFIA [7] verifies CBC-MACs over blocks of four to six instructions, and CONFIDAENT [25] applies ASCON with 64-bit tags to 16-byte blocks plus mask-based control-flow integrity (CFI) requiring 256 bits of metadata per block. Both schemes confine detection to block boundaries and require dedicated compiler support. Implicit single-layer schemes instead chain the cipher state through the instruction stream: SCFP [36] places a sponge-based decrypt stage between fetch and decode, and Gousselot et al. [10] extend this philosophy with an ASCON duplex in the CV32E40P fetch stage that binds the pipeline's control signals. Chaining buys CFI at low cost (SCFP reports 9.1% runtime and 19.8% code size [36]), but detection lacks determinism: no tag is generated, and a tampered word is caught only once the diverging keystream decodes into an illegal opcode, which may be after tens of instructions [10]. Chaining schemes necessitate deep coupling with program semantics: whole-program control-flow graph analysis, per-edge patching or ISA extensions, and pipeline-specific control-signal modelling [10, 36]. Ahmad et al. [1] use explicit per-instruction HMACs to prevent replay and control-flow hijacking, but combines separate encryption and authentication layers, and it is not evaluated in hardware.

Positioning. Sable uses a single authenticated encryption with associated data (AEAD) primitive with instruction independence, so a failed verification raises a deterministic hardware fault in the fetch cycle, rather than after several pseudo-random instructions may have retired. Sable does not require CFG analysis, binary patching, nor compiler modification besides a modified `objcopy`, and ports across micro-architectures. We do not claim to exceed other scheme' performance: per-instruction tags cost Sable 4.1–10× runtime and 5× IMEM where SCFP costs ≈9% and ≈20% [36]. That reflects a 64-bit PRINCE state; instantiated with ASCON's 320-bit state, the same chained construction costs roughly 10× IMEM

Table 1: Positioning of Sable against related hardware-assisted instruction-protection schemes.

Scheme	Primary threat	Primitive	Granularity	ISA	Auth.	Pre-exec. detection	Single layer	Analy. -free	Validation
ASIST [22]	Code injection	XOR / transposition	Instr. / block (enc.)	SPARC V8	✗	✗	✓	✓	FPGA
Morpheus [8]	Control-flow hijacking	QARMA-64 + displacement	64-bit word (enc.)	RISC-V	✗	✗	✗	▲	Sim. (gem5)
Morpheus II [11]	Remote code exec.	SIMON (12-round)	Instr. + code ptr. (enc.)	*RISC-V	✗	✗	✓	✗	FPGA
Lee et al. [16]	Counterfeiting, IP theft	Chained LFSR masking	Instruction (enc.)	PicoBlaze	✗	✗	✓	✓	FPGA
SEP [17]	Tampering, counterfeiting	PRINCE (addr.-keyed, chained)	Block (4 instr.)	Custom	▲	▲	✗	✗	FPGA
SOFIA [7]	Code tampering, reuse	Enc. + CBC-MAC	Block (4–6 instr.)	SPARC V8	✓	✓	✗	✗	FPGA
CONFIDAENT [25]	Faults, code reuse	ASCON + masking	Block (16 B)	*RISC-V	✓	✓	✗	✗	FPGA
SCFP [36]	Faults, code reuse	Sponge AE (chained)	Instruction	*RISC-V	▲	▲	✓	✗	ASIC
Gousselot et al. [10]	Faults, control signals	ASCON duplex (chained)	Instruction	RISC-V	▲	▲	✓	✗	FPGA
Ahmad et al. [1]	Code tampering	Enc. + HMAC (chained)	Instruction	RISC-V (sim.)	✓	✓	✗	✗	Python
Sable	Injection, tampering, IP theft	ASCON (Design space exploration)	Instruction	RISC-V	✓	✓	✓	✓	FPGA

Selected column details. **Granularity**: the smallest unit of operation for encryption and authentication. **Auth.**: cryptographic authentication is provided. **Pre-exec. detection**: unauthorised instructions are detected prior to execution. **Single layer**: a single cryptographic mechanism provides all claimed protection. **Analy.-free**: no program semantic analysis, binary patching, or ISA modification is required. *Symbols.* ✓ = provided; for detection, deterministic and before the instruction executes. ▲ = partial or implicit; detection yields pseudo-random opcodes that may execute. ✗ = absent. * = Requires non-standard instructions added to the base ISA and corresponding compiler support.

in patches and a 30–72% clock-frequency reduction [10], placing Sable within the established cost of ASCON-class protection. We consider this appropriate where deterministic pre-execution detection and toolchain independence are preferred. Nor do we claim CFI, which Sable deliberately forgoes. The hardware cost space of this approach remains largely uncharted, which we address, and no prior scheme reports power or energy, or integration points.

2 Proposed Designs

We now explore the design space of integrating authenticated ILE into an embedded processor. First, single-cycle implementations of ASCON-128a decryption are presented, with and without authentication, and with their integration into IMEM and CPU frontend of the reference (NEORV32) architecture. Second, multi-cycle implementations are discussed, and their impact on performance, timing, energy and system integration.

2.1 Single-Cycle Decryption Architectures

Sable employs a lightweight cryptography module introduced, firstly, at the instruction fetch stage. ASCON-128a operates on a 320-bit internal state of five 64-bit words in a sponge construction; its core permutation combines round-constant addition and a 5-bit substitution layer; the algorithm takes a 128-bit key and 128-bit public nonce and produces a 128-bit authentication tag. We develop a single-cycle VHDL implementation of ASCON-128a decryption specialised for a fixed 32-bit ciphertext (one RISC-V instruction) and zero-length associated data. Under these constraints, this enables a fully unrolled, straight-line combinational datapath organised as (i) an initialisation stage loading the IV, key, and nonce into the state; (ii) a fully unrolled 12-round `ASCON-p[12]` permutation; and (iii) a decryption stage producing the plaintext instruction. To synthesise the fully combinational permutation, it is modularised into a dedicated VHDL entity with explicit inter-round signals annotated `DONT_TOUCH`, preventing aggressive optimisation. A tagless (confidentiality-only) variant is also explored.

For authentication, a 128-bit tag is produced and compared with the expected one, yielding a binary signal (`tag_ok`). If it fails, the decrypted output is replaced with an illegal instruction and a hardware exception is asserted, preventing the execution of unauthorised instructions. The full 128-bit ASCON-128a tag is retained per instruction word; this memory cost is a deliberate trade-off in favour of an unreduced integrity margin rather than a consequence of the tag size. This requires 5× the instruction memory capacity compared to the baseline processor and the design employing tagless decryption. For this baseline, we use NEORV32 [21], a compact 32-bit RISC-V soft-core with a multi-cycle, finite-state machine (FSM)-based frontend (instruction fetch and buffering) and backend (decode and execution), decoupled by an Instruction Prefetch Buffer (IPB), with coupled instruction and data memories on a memory-mapped bus. We integrate the decryptor at two points: the IMEM and the CPU frontend.

2.1.1 IMEM-Level Integration. In our baseline design (`IMDEC`), a single-cycle tagless decryptor is inserted into the IMEM read path (`neorv32_imem.vhd`), transparently decrypting all instruction and read-only-data accesses within a single clock cycle, before they reach the processor bus and with no protocol changes. The key and a base nonce are stored in dedicated write-only registers; the per-instruction nonce is formed by adding the 32-bit fetch address into the lowest 32 bits of the 128-bit base nonce, binding each ciphertext to its memory location so that a valid (ciphertext, tag) pair cannot be relocated or spliced to a different address (Eq. 1).

$$N(a) \;=\; N_{\text{base}}[127{:}32] \;\|\; \big((N_{\text{base}}[31{:}0] + (a \gg 2)) \bmod 2^{32} \big) \quad (1)$$

This approach offers minimal architectural intrusion at the cost of critical-path delay. The timing path spans the memory array, the full combinational ASCON datapath, and the bus output, limiting the maximum operating frequency. We also develop an authenticated variant (`IMAUDC`) that adds a dedicated Tag Memory (TAGMEM)—a synthesis-time-initialised VHDL package storing one 128-bit tag/output per instruction word, accessed synchronously with IMEM under the same address and directly connected to the cryptographic module. TAGMEM's contents are not confidential: because its entries are ASCON-128a tags, an adversary cannot produce a matching tag for a chosen instruction without the key. The tags may reside in the same untrusted memory as the encrypted code as an optional design consideration. The bus is driven with the plaintext instruction only when `tag_ok` holds; otherwise an illegal instruction is injected and a bus error raised, preventing execution of tampered code without modifying the processor core.

Table 2: Multi-cycle SABLE designs.

	MCP12	MCP6	MCP4	MCP3	MCP2	MCP1
ASCON rounds / cycle	12	6	4	3	2	1
Cycles / ASCON perm.	1	2	3	4	6	12
Time-mux. cycles	2	4	6	8	12	24
FSM states	5	7	9	11	15	27
Added latency per inst. (cycles)	7	9	11	13	17	29

2.1.2 Frontend-Level Integration. We develop an encryption-only design variant (FEDEC) that places the decryptor between the instruction fetch and IPB stages. The frontend operates as an FSM with three states: RESTART, REQUEST, and PENDING, managing fetching and control-flow changes. In a second authenticated design variant (FEAUDC), we add a frontend TAGMEM indexed by the fetch address, whose 128-bit output feeds an ASCON module directly. Only instructions with a valid tag are written into the IPB; otherwise an illegal instruction is injected and an instruction-fetch error raised. Frontend integration limits the exposure of decrypted instructions and provides fine-grained control over instruction validation while maintaining compatibility with the existing NEORV32 pipeline.

2.2 Multi-Cycle Architectures

The single-cycle authenticated design instantiates two ASCON-p[12] permutations—one for decryption, one for tag generation and verification—resulting in a long critical path and high area cost. Our multi-cycle family addresses this by folding: partitioning ASCON-p[12] into r rounds per cycle and reusing the logic across cycles shortens the critical path and shares resources, at the cost of a higher cycle count per instruction (CPI). Six designs, MCP12 through MCP1, implement 12 down to 1 round(s) per cycle; Table 2 summarises the resulting permutation latency, FSM complexity, and added instruction latency. All designs share a common interface (rstn, start, clk, done) with input and output registers for ciphertext, plaintext, and tag_ok; an FSM sequences the time-multiplexed permutation passes, split evenly between decryption and authentication. The rstn signal initialises the internal FSM to an IDLE state, while assertion of start triggers the decryption and authentication process. Upon tag verification, the done signal is asserted for one clock cycle. The multi-cycle decryptors are integrated into the NEORV32 CPU frontend by modifying its three-state fetch FSM (RESTART, REQUEST, PENDING): RESTART asserts the decryptor reset; PENDING loads the acknowledged ciphertext into the decryptor's input register and asserts start; and a new wait state holds until done is asserted, upon which the plaintext is written to the IPB if tag_ok holds, otherwise an illegal instruction machine code is written to the IPB and an exception is triggered. The frontend updates the program counter and resumes normal control flow. This integration enables a trade-off between performance and hardware cost, allowing designers to select an appropriate architecture based on application constraints.

3 Toolchain and Verification

A design goal of SABLE is that programs are built with the standard flow, and encryption and tag generation are post-link transformations over the resulting binaries (Figure 2). SABLE requires fixed-width 32-bit instructions (Section 2.1). A GCC toolchain targeting

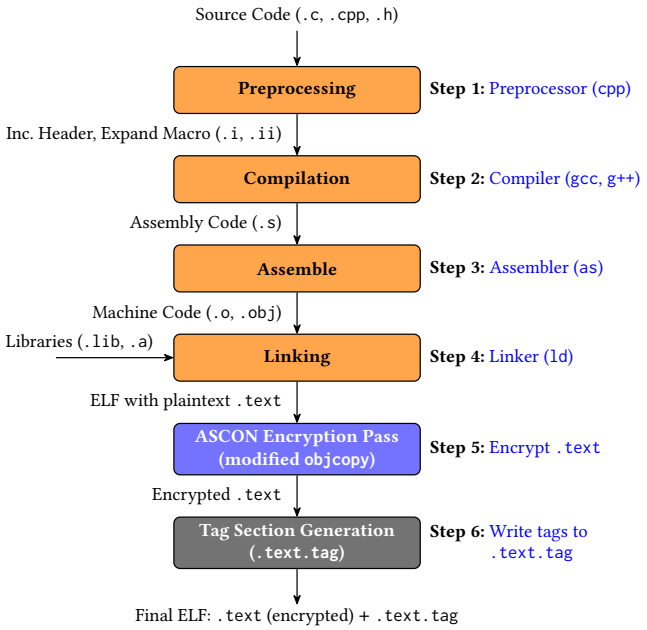


Figure 2: Modified RISC-V toolchain with instruction (.text) encryption and authentication tag generation (in .text.tag).

rv32im was built with compressed (16-bit) instructions disabled. Applications and standard libraries were compiled with it; the absence of compressed instructions was verified by disassembly across the software stack. Two back-ends then apply the same per-word ASCON-128a encryption to the linked output.

ELF back-end. A modified objcopy post-processes compiled ELF binaries, encrypting the .text section in place as independent 4-byte ciphertext blocks and leaving the binary's layout untouched. Authenticated mode additionally stores one 16-byte ASCON-128a tag per instruction in a dedicated .text.tag section while keeping .text at its original density; this requires no branch patching, preserves ISA compliance, permits parallel power-of-two-aligned fetches of instructions and tags, and confines all cryptographic logic to a single toolchain file with no BFD dependencies. A custom linker script defines two separate PT_LOAD program headers—one for .text (read/execute) and one for .text.tag (read-only)—and reserves address space for the tag section as a NOBITS placeholder of size SIZEOF(.text) × 4, i.e., one 16-byte tag per 4-byte instruction, realising the 5× instruction-memory cost established in Section 2.1. The modified objcopy then performs two passes: it encrypts each instruction in place, collects the resulting tags into a separate buffer, writes the encrypted instructions back into .text, and writes the tag buffer into .text.tag, which the BFD library promotes from NOBITS to PROGBITS; section-flag corrections in setup_section() ensure .text.tag carries correct allocate-only flags.

FPGA back-end. For FPGA targets, an offline C++ tool applies the same per-word encryption directly to the NEORV32 instruction-memory image (neorv32_imem_image.vhd), emitting in a single pass the encrypted IMEM image and a matching TAGMEM package (encryption-only designs omit the latter). The back-ends differ only in scope: IMEM-level integration encrypts both instructions and

read-only data (`.rodata`), whereas frontend integration encrypts only the instruction section (`.text`).

3.1 Spike Functional Reference Model

To provide a reference independent of the RTL, we extended the Spike RISC-V ISA simulator [35] with ASCON-128a support from the reference C implementation [30], first for encryption-only execution and then for authenticated execution. Spike's ELF loader records the encrypted region's virtual-address bounds, and the MMU's `refill_icache()` invokes ASCON-128a decryption on a cache miss for fetches within it, before decode. Unencrypted binaries remain backward-compatible; decryption activates only when a valid encrypted `.text` region is detected at load time. In authenticated mode, Spike maps `.text.tag` into simulated memory as an ordinary `PT_LOAD` segment, requiring no loader modification. The fetch unit locates the tag of any instruction using the address formula `tag_addr = TAG_BASE + ((PC - TEXT_BASE) » 2) × 16`, where `TAG_BASE` is read dynamically from the `_tag_start` ELF symbol at load time, reducing tag addressing to a single shift and add. Upon a cache miss, the fetch unit retrieves the 4-byte encrypted instruction and its corresponding 16-byte tag, invokes ASCON-128a decryption with authentication, and raises a fetch fault/illegal opcode if tag verification fails—mirroring, the pre-decode fault semantics of the hardware designs (ISA level) (Section 2.1).

3.2 Experimental Setup and Verification

Functional verification used the modified Spike RISC-V ISA simulator (version 1.1.1), with GNU Binutils 2.46, including the GNU assembler and `objcopy`, and GCC 15.2.0 (`riscv32-unknown-elf`). Our hardware prototype targets the Digilent Nexys A7-100T board (Xilinx Artix-7 XC7A100T), with on-chip block RAM (BRAM) for code and data and UART for runtime interaction, synthesised and implemented with Vivado ML Edition 2025.2. Two C++ suites were used across simulation and hardware: `hello_world` (610 static instructions in `.text`; 822 executed) and Dhrystone with 10,000 iterations (1,214 static in `.text`; 4,541,606 executed). On the baseline processor these require 3,995 and 18,796,356 clock cycles, respectively, excluding input/output cycles.

The flow was verified end-to-end at four levels. At the software level, a companion C++ tool performed decryption of every 4-byte ciphertext with its 16-byte tag and matched the original plaintext image bit-exactly, with zero authentication failures. At the ISA level, the encrypted, authenticated binaries executed correctly under the extended Spike, exercising the loader, tag addressing, and authenticated-fetch path independently of the RTL. At the simulation level, GHDL (VHDL-2008) runs of encrypted, authenticated `hello_world` and Dhrystone confirmed correct decryption and authentication across all 822 and 4,541,606 executed instruction words, respectively, reproducing the expected UART output without errors. In hardware, the designs were synthesised, implemented and executed on the target FPGA. These checks were complemented by negative testing: deliberate corruption of instruction words in IMEM and tags in TAGMEM caused every affected word to fail authentication, deasserting `tag_ok` and correctly raising an illegal instruction exception in both GHDL simulation and on the FPGA.

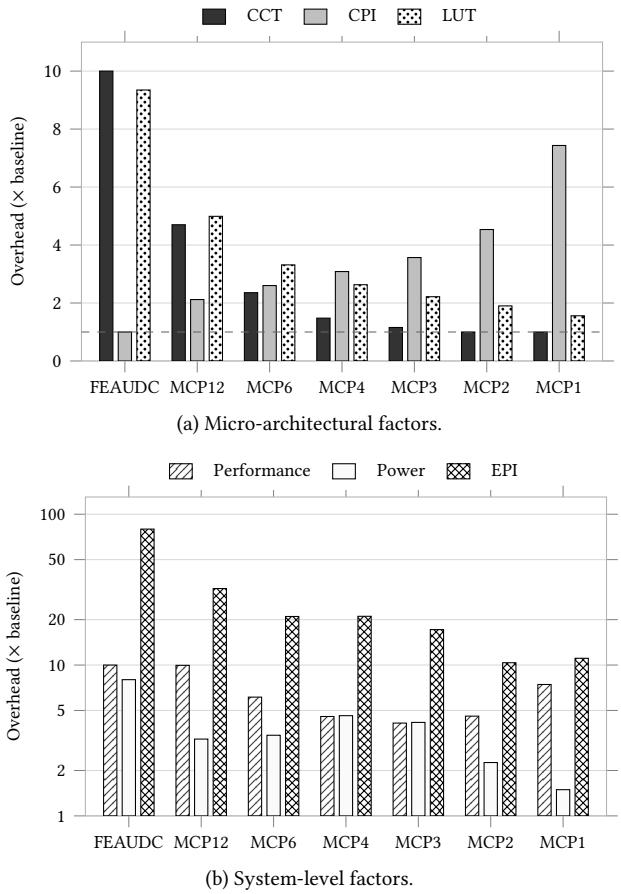


Figure 3: Normalised metrics relative to the NEORV32 baseline using Dhrystone. The dashed line in (a) marks 1×.

4 Evaluation

4.1 Synthesis Results

Table 3 presents synthesis results for the baseline NEORV32 alongside the single-cycle ASCON-128a decryptors in tagless and authenticated configurations. The evaluated NEORV32 instance is a single-core design with the compressed-instruction (C) and code-size-reduction (Zcb) extensions disabled and 32 KB/64 KB instruction/data memories. The ASCON dominates hardware cost in both configurations, driving area, critical-path delay, and power. Notably, power consumption of the authenticated design is ≈4.4× the baseline, while LUT usage is ≈8.3× NEORV32.

4.2 Single-Cycle Design Results

Table 4 compares the four single-cycle secure configurations against the baseline. The four configurations are: (1) tagless ASCON-128a decryption integrated into the instruction memory wrapper (`IMDEC`); (2) fully authenticated ASCON-128a decryption integrated into the instruction memory wrapper (`IMAUDC`); (3) tagless ASCON-128a decryption integrated into the CPU frontend (`FEDEC`); and (4) fully authenticated ASCON-128a decryption integrated into the CPU frontend (`FEAUDC`). The capability rows expose a security-relevant

Table 3: Synthesis results.

Design	LUTs	FFs	Power (mW)	Delay (ns)	Block RAM Tile
NEORV32 (Base)	1867	1428	120	9	19
ASCON-128a (Enc.-only)	7590	0	336	29.8	0
ASCON-128a (Auth.)	15621	0	533	88	0

Table 4: Single-cycle design results (performance overhead given as $N\times$ vs. NEORV32 baseline).

Metrics	IMDEC	IMAUDC	FEDEC	FEAUDC
Authentication	✗	✓	✗	✓
Clock Cycle Time	5.4×	10.2×	4.9×	10×
CPI	1×	1×	1×	1×
LUTs Overhead	4.8×	9.3×	4.8×	9.3×
Performance Overhead	5.4×	10.2×	4.9×	10×
Power Overhead	4.3×	7.9×	4.3×	8×

distinction: IMEM-level integration additionally protects read-only data but places plaintext on the processor bus, whereas frontend integration keeps ciphertext on the bus, denying the instruction stream to an attacker with bus-monitoring capability. CPI is invariant across all configurations, since the fully combinational (authenticated) decryption completes within a single, elongated clock cycle and introduces no stalls or wait states; each configuration's performance overhead therefore equals its clock-cycle-time ratio to the baseline. Adding authentication roughly doubles area and delay over tagless decryption, as two ASCON 12-round permutation instances are required, and `IMAUDC`/`FEAUDC` each consume 8 additional BRAM tiles for the TAGMEM holding one 128-bit tag per instruction word. The baseline was synthesised and implemented under a 100 MHz timing constraint; the four single-cycle secure configurations under 10 MHz, owing to the ASCON critical path.

4.3 Multi-Cycle Design Results

Figure 3 characterises the multi-cycle design space using Dhrystone for `FEAUDC`—the single-cycle upper bound of the design space—and the six multi-cycle configurations, normalised to the NEORV32 baseline. Figure 3(a) reports the three microarchitectural factors: clock cycle time (CCT), CPI, and LUT count. Figure 3(b) reports the three system-level overheads: performance, power, and energy per instruction (EPI). Because the executed instruction count is invariant across configurations while both the clock period and the CPI vary, the performance overhead of each configuration is the product of its normalised CCT and CPI; EPI is in turn the product of the normalised power and performance overheads. EPI is invariant to the per-design clock frequency, enabling the comparison of configurations synthesised under different timing constraints.

Each design was implemented at the highest clock frequency at which it closed timing: the baseline, `MCP1`, and `MCP2` at 100 MHz; `MCP3` at 90 MHz; `MCP4` at 70 MHz; `MCP6` at 25 MHz; and `MCP12` and `FEAUDC` at 10 MHz, reflecting their progressively longer critical paths. Power estimates are activity-aware: switching activity captured during post-implementation functional simulation was exported in the Switching Activity Interchange Format (SAIF) and annotated into the power-analysis flow, and each design's clock period derives from its post-implementation critical-path delay. Since the configurations are therefore synthesised under different timing constraints, raw power is not directly comparable across designs; the EPI metric normalises for this.

Figure 3(a) exposes the central trade-off of the design space. Folding the ASCON-p[12] permutation across more clock cycles places fewer rounds, and hence less combinational logic, in each stage, so the critical path shortens monotonically (CCT falls from 10× at `FEAUDC` to 1× at `MCP2` and `MCP1`) and the decryptor shrinks (from 9.3× to 1.6× LUTs), while the CPI rises monotonically (from 1× to 7.4×) as authenticated decryption completes over a greater number of cycles. The overheads in Figure 3(b) show that no single configuration dominates: the design space contains three distinct optima. `MCP1` minimises area (1.6× LUTs), power (1.5×), and CCT (1×); `MCP3` minimises performance overhead (4.1×); and `MCP2` minimises EPI overhead (10.4×). Moreover, folding the permutation lowers EPI by roughly an order of magnitude relative to the single-cycle `FEAUDC` (79.7×), demonstrating that the multi-cycle designs are smaller and fundamentally more energy-efficient per executed instruction.

4.4 Security Analysis

Sable draws the secrecy of stored code directly from ASCON-128a under a CPU-bound key that is never exposed. Nonce uniqueness is required for ASCON-128: within a single code image, the base-nonce-plus-address construction ensures a distinct nonce per instruction word. Across re-encryptions (e.g., firmware updates) each image must be produced under a fresh key or fresh base nonce, which we assume the provisioning process enforces. Identical instructions, and instructions sharing fields such as opcodes, thus yield unrelated ciphertexts. An adversary confined to the encrypted memory is unable to access plaintext code.

Nevertheless, an adversary who cannot read the code can still modify it, and a corrupted word may itself decode into an executable instruction. Moreover, encrypted IMEM does not hide which addresses are fetched, so access and timing patterns may still leak coarse program structure. The latter is outside our scope; the former is closed by binding each ciphertext word to its stored address through the address-derived nonce and to a tag checked as the word is decrypted. Any change to the ciphertext, its tag, or its location makes the check fail, and an exception is raised deterministically before it is executed. This is a sharper guarantee than the implicit schemes [10, 36], where a tampered word decrypts to a pseudo-random value caught only when it decodes illegally, by which point several instructions may already have executed. Bit-flips in IMEM or TAGMEM fail authentication and halt execution; injected code carries no valid tag and is rejected before decode, while relocation and splicing fail because the nonce is tied to the fetch address. Sable also defeats the Cipher Instruction Search Attack [15]: since no ciphertext can verify without the key, an adversary cannot iteratively probe the decryptor to assemble a working instruction.

5 Conclusion and Future Work

We introduced a design exploration of Sable, which performs instruction-level authenticated decryption on RISC-V, protecting

stored code against injection, tampering, and static reverse engineering threats. We presented and integrated seven decryptor implementations using ASCON-128a—single-cycle down to one round per cycle—at the NEORV32 IMEM wrapper and CPU frontend in tagless and authenticated configurations, which were validated in an extended Spike ISA simulator, in GHDL simulation, and on an Artix-7 FPGA. Relative to the baseline, LUT, performance, power, and EPI overheads span 9.3×–1.6×, 10×–4.1×, 8×–1.5×, and 80×–10.4×, with authenticated execution requiring 5× instruction memory for per-instruction 128-bit tags. Explicit, pre-execution authenticated execution is thus feasible on lightweight embedded RISC-V cores, and this work constitutes, to our knowledge, the first FPGA-validated power–performance–area–energy design-space exploration of instruction-level ASCON-128a authenticated decryption. Future areas of research include supporting CFI enforcement, authenticated encryption of data memory, and compressed-instruction support.

Acknowledgments

This work was supported by the UK Engineering and Physical Sciences Research Council via the 'Chameleon' project (EP/Y030168/1).

References

[1] Shahzad Ahmad, Stefan Rass, Maksim Goman, Manfred Schlägl, and Daniel Große. 2025. Control Flow Protection by Cryptographic Instruction Chaining. In *Proc. 22nd Int'l Conf. Security and Cryptography (SECRYPT)*.

[2] Roberto Avanzi. 2017. The QARMA Block Cipher Family. Almost MDS Matrices Over Rings With Zero Divisors, Nearly Symmetric Even-Mansour Constructions With Non-Involutory Central Rounds, and Search Heuristics for Low-Latency S-Boxes. *IACR Transactions on Symmetric Cryptology* 2017, 1 (2017), 4–44.

[3] Nathan Binkert, Bradford Beckmann, Gabriel Black, Steven K. Reinhardt, Ali Saidi, Arkaprava Basu, Joel Hestness, Derek R. Hower, Tushar Krishna, Somayeh Sardashti, et al. 2011. The gem5 Simulator. *ACM SIGARCH Computer Architecture News* 39, 2 (2011), 1–7.

[4] Otto Bittner, Thilo Krachenfels, Andreas Galauner, and Jean-Pierre Seifert. 2021. The Forgotten Threat of Voltage Glitching: A Case Study on Nvidia Tegra X2 SoCs. In *Workshop on Fault Detection and Tolerance in Cryptography*. IEEE.

[5] Tyler Bletsch, Xuxian Jiang, Vince W Freeh, and Zhenkai Liang. 2011. Jump-oriented programming: A new class of code-reuse attack. In *Proc. 6th ACM Symposium on Information, Computer and Communications Security*. 30–40.

[6] George Christou, Giorgos Vasiliadis, Vassilis Papaefstathiou, Antonis Papadogiannakis, and Sotiris Ioannidis. 2020. On Architectural Support for Instruction Set Randomization. *ACM Trans. Archit. Code Optim.* 17, 4 (2020).

[7] Ruan de Clercq, Johannes Götzfried, David Übler, Pieter Maene, and Ingrid Verbauwhede. 2017. SOFIA: Software and control flow integrity architecture. *Computers & Security* 68 (2017), 16–35.

[8] Mark Gallagher, Lauren Biernacki, Shibo Chen, Zelalem Birhanu Aweke, Salessawi Ferede Yitbarek, Misiker Tadesse Aga, Austin Harris, Zhixing Xu, Baris Kasikci, Valeria Bertacco, et al. 2019. Morpheus: A Vulnerability-Tolerant Secure Architecture Based on Ensembles of Moving Target Defenses with Churn. In *Proc. Int'l Conf. 24th Architectural Support for Programming Languages and Operating Systems*. ACM.

[9] Yang Gao, Siqi Lu, Yongjuan Wang, Haopeng Fan, Qingdi Han, and Jingsheng Li. 2025. ECP: Coprocessor Architecture to Protect Program Logic Consistency. *Journal of Software: Evolution and Process* 37, 4 (2025).

[10] Théophile Gousselot et al. 2025. Code Encryption for Confidentiality and Execution Integrity Down to Control Signals. In *IEEE Int'l Symposium on Hardware Oriented Security and Trust (HOST)*. IEEE.

[11] Austin Harris, Tarunesh Verma, Shijia Wei, Lauren Biernacki, Alex Kisil, Misiker Tadesse Aga, Valeria Bertacco, Baris Kasikci, Mohit Tiwari, and Todd Austin. 2021. Morpheus II: A RISC-V Security Extension for Protecting Vulnerable Software and Hardware. In *IEEE Int'l Symposium on Hardware Oriented Security and Trust (HOST)*.

[12] Thomas Hiscock, Olivier Savry, and Louis Goubin. 2019. Lightweight Instruction-Level Encryption for Embedded Processors Using Stream Ciphers. *Microprocessors and Microsystems* 64 (2019), 43–52.

[13] Yuchen Jiang, Shimeng Wu, Renjie Ma, Ming Liu, Hao Luo, and Okyay Kaynak. 2023. Monitoring and defense of industrial cyber-physical systems under typical attacks: From a systems and control perspective. *IEEE Trans. Industrial Cyber-Physical Systems* 1 (2023), 192–207.

[14] Stephan Kleber, Florian Unterstein, Matthias Matousek, Frank Kargl, Frank Slomka, and Matthias Hiller. 2015. Secure Execution Architecture based on PUF-driven Instruction Level Code Encryption. Cryptology ePrint Archive, Paper 2015/651. https://eprint.iacr.org/2015/651

[15] M. G. Kuhn. 1998. Cipher Instruction Search Attack on the Bus-Encryption Security Microcontroller DS5002FP. *IEEE Trans. Computers* 10 (1998).

[16] Robert P. Lee, Konstantinos Markantonakis, and Raja Naeem Akram. 2016. Binding Hardware and Software to Prevent Firmware Modification and Device Counterfeiting. In *Proc. 2nd ACM Int'l Workshop on Cyber-Physical System Security*.

[17] Robert P. Lee, Konstantinos Markantonakis, and Raja Naeem Akram. 2019. Ensuring Secure Application Execution and Platform-Specific Execution in Embedded Devices. *ACM Transactions on Embedded Computing Systems* 18, 3 (2019), 1–21.

[18] Alyssa Milburn, Niek Timmers, Nils Wiersma, Ramiro Pareja, and Santiago Cordoba. 2018. There will be glitches: Extracting and analyzing automotive firmware efficiently. *Black Hat USA* (2018).

[19] Mohammad Monjur, Joshua Calzadillas, and Qiaoyan Yu. 2023. Hardware security risks and threat analyses in advanced manufacturing industry. *ACM Trans. Design Automation of Electronic Systems* 28, 5 (2023), 1–22.

[20] Lionel Morel, Damien Courossé, and Thomas Hiscock. 2023. Code Polymorphism Meets Code Encryption: Confidentiality and Side-Channel Protection of Software Components. *Digital Threats: Research and Practice* 4, 2 (2023), 1–27.

[21] Stephan Nolting. 2020. NEORV32: A Small, Customizable and Extensible MCU-Class 32-bit RISC-V Soft-Core CPU and Microcontroller-Like SoC. https://github.com/stnolting/neorv32. Accessed: 2026-07-16.

[22] A. Papadogiannakis, L. Loutsis, V. Papaefstathiou, and S. Ioannidis. 2013. ASIST: Architectural Support for Instruction Set Randomization. In *Proc. ACM Conf. Computer and Communications Security*. 981–992.

[23] Enrico Pozzobon, Nils Weiß, Jürgen Mottok, and Vaclav Matousek. 2023. Fuzzy fault injection attacks against secure automotive bootloaders. In *21th ESCAR European Automotive Cyber Security Conference*. doi:10.13154/294-10381

[24] Ryan Roemer, Erik Buchanan, Hovav Shacham, and Stefan Savage. 2012. Return-Oriented Programming: Systems, Languages, and Applications. *ACM Transactions on Information and System Security* 15, 1 (2012). doi:10.1145/2133375.2133377

[25] Olivier Savry, Mustapha El-Majihi, and Thomas Hiscock. 2020. CONFIDAENT: Control Flow Protection with Instruction and Data Authenticated Encryption. In *23rd Euromicro Conf. Digital System Design*. IEEE.

[26] Carlton Shepherd, Konstantinos Markantonakis, Nico Van Heijningen, Driss Aboulkassimi, Clément Gaine, Thibaut Heckmann, and David Naccache. 2021. Physical fault injection and side-channel attacks on mobile devices: A comprehensive analysis. *Computers & Security* (2021).

[27] Omer Shwartz, Yael Mathov, Michael Bohadana, Yuval Elovici, and Yossi Oren. 2017. Opening Pandora's box: effective techniques for reverse engineering IoT devices. In *Int'l Conf. Smart Card Research and Advanced Applications*. Springer.

[28] Kanad Sinha, Vasileios P. Kemerlis, and Simha Sethumadhavan. 2017. Reviving Instruction Set Randomization. In *Proc. IEEE Int'l Symposium on Hardware Oriented Security and Trust (HOST)*. IEEE, 21–28.

[29] M. M. Tehranipoor, U. Guin, and S. Bhunia. 2017. Invasion of the Hardware Snatchers. *IEEE Spectrum* 54, 5 (2017), 36–41. doi:10.1109/MSPEC.2017.7906898

[30] The Ascon Team. 2026. ascon-c: Reference, Highly Optimized, Masked C and ASM Implementations of Ascon (NIST SP 800-232). https://github.com/ascon/ascon-c. Accessed: 2026-07-16.

[31] Minh Tran, Mark Etheridge, Tyler Bletsch, Xuxian Jiang, Vincent Freeh, and Peng Ning. 2011. On the expressiveness of return-into-libc attacks. In *Int'l Workshop on Recent Advances in Intrusion Detection*. Springer, 121–141.

[32] Shahid Ul Haq, Yashwant Singh, Amit Sharma, Rahul Gupta, and Dipak Gupta. 2023. A survey on IoT and embedded device firmware security: Architecture, extraction techniques, and vulnerability analysis frameworks. *Discover Internet of Things* 3, 1 (2023), 17. doi:10.1007/s43926-023-00045-2

[33] Jan Van den Herrewegen, David Oswald, Flavio D. Garcia, and Qais Temeiza. 2021. Fill your Boots: Enhanced Embedded Bootloader Exploits via Fault Injection and Binary Analysis. *IACR Transactions on Cryptographic Hardware and Embedded Systems* (2021).

[34] Sebastian Vasile, David Oswald, and Tom Chothia. 2018. Breaking all the things—A systematic survey of firmware extraction techniques for IoT devices. In *Int'l Conf. Smart Card Research and Advanced Applications*. Springer, 171–185.

[35] Andrew Waterman, Yunsup Lee, et al. 2011. Spike: A RISC-V ISA Simulator. https://github.com/riscv-software-src/riscv-isa-sim. Accessed: 2026-07-16.

[36] Mario Werner, Thomas Unterluggauer, David Schaffenrath, and Stefan Mangard. 2018. Sponge-Based Control-Flow Protection for IoT Devices. In *IEEE European Symposium on Security and Privacy*. IEEE.

[37] Tahreem Yaqoob, Haider Abbas, and Mohammed Atiquzzaman. 2019. Security vulnerabilities, attacks, countermeasures, and regulations of networked medical devices—A review. *IEEE Communications Surveys & Tutorials* 21, 4 (2019).

[38] Y. Zhang, Y. Li, and Z. Li. 2023. Aye: A Trusted Forensic Method for Firmware Tampering Attacks. *Symmetry* 15 (2023), 145. doi:10.3390/sym15010145